

C Functions

A function in C is a set of statements that when called perform some specific tasks. The programming statements of a function are enclosed within { } braces, having certain meanings and performing certain operations.

Every C program includes at least one function, typically the main () function, which serves as the entry point for execution. Functions can take parameters and return values, allowing them to be versatile and reusable across different parts of a program. They are also called subroutines or procedures in other languages.

Importance of Functions in C

1. Modularity

Functions promote modular programming by dividing a program into distinct sections. Each function can handle a specific task, making the overall program structure clearer and more organized. This modularity simplifies both development and debugging processes.

2. Reusability

Once defined, functions can be reused multiple times throughout a program without needing to rewrite code. This not only saves time but also reduces redundancy, leading to more efficient coding practices.

3. Readability

By breaking complex code into smaller functions, programs become easier to read and understand. This is particularly beneficial for larger projects where maintaining clarity is crucial for collaboration among multiple developers.

4. Easier Debugging

Functions help isolate specific tasks within a program, making it easier to identify and fix errors. When issues arise, developers can focus on individual functions rather than sifting through extensive code.

5. Abstraction

Functions provide a level of abstraction by encapsulating specific functionalities. This means that users of the function do not need to understand its internal workings; they only need to know how to call it and what it returns.

6. Efficiency

Functions can be optimized individually, which can lead to overall performance improvements in the program. By focusing on optimizing specific functions rather than the entire program at once, developers can achieve better efficiency.

Types of Functions:

1. Library Function

A library function is also referred to as a “built-in function”. A compiler package already exists that contains these functions, each of which has a specific meaning and is included in the package. Built-in functions have the advantage of being directly usable without being defined, whereas user-defined functions must be declared and defined before being used.

For Example:

`pow()`, `sqrt()`, `strcmp()`, `strcpy()` etc.

Example:

```
#include <math.h>
#include <stdio.h>
// Driver code
int main(){
double Number;
Number = 49;
// Computing the square root with the help of predefined C library function
double squareRoot = sqrt(Number);
printf("The Square root of %.2lf = %.2lf", Number, squareRoot);
return 0;}
```

Output

The Square root of 49.00 = 7.00

2. User Defined Function

Functions that the programmer creates are known as User-Defined functions or “tailor-made functions”. User-defined functions can be improved and modified according to the need of the programmer. User-defined functions can be further categorized based on their parameters and return values:

1. **Functions with Arguments and Return Values:** These functions take parameters and return a value. They are often used for calculations or transformations.

2. **Functions with Arguments and Without Return Values:** These functions accept parameters but do not return any value, often used for operations that do not require a result to be sent back.
3. **Functions without Arguments and With Return Values:** These functions do not take any parameters but return a value, useful for simple computations that do not require input.
4. **Functions without Arguments and Without Return Values:** These functions neither take parameters nor return values, typically used for performing an action without needing input or output.

// C program to show user-defined functions

```
#include <stdio.h>
```

```
int sum(int a, int b) {  
    return a + b; }
```

// Driver code

```
int main(){  
    int a = 30, b = 40;  
    // function call  
    int res = sum(a, b);  
    printf("Sum is: %d", res);  
    return 0;}
```

Output

Sum is: 70

Function Declarations:

A function declaration serves as a promise to the compiler that a function exists with a specified name, return type, and parameters. It does not include the actual implementation of the function.

The primary purpose of declaring a function is to inform the compiler about its signature so that it can be called before its definition appears in the code.

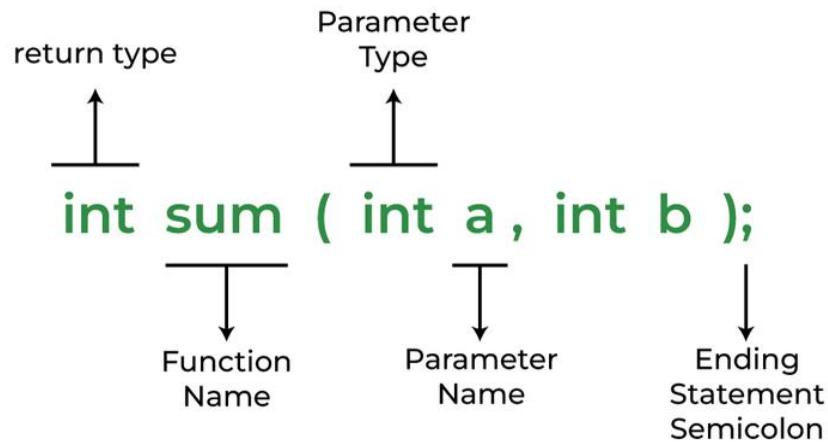
Syntax

```
return_type name_of_the_function (parameter_1, parameter_2);
```

The parameter name is not mandatory while declaring functions. We can also declare the function without using the name of the data variables.

Example

```
int sum(int a, int b); // Function declaration with parameter names
int sum(int , int);    // Function declaration without parameter names
```



Note: A function in C must always be declared globally before calling it.

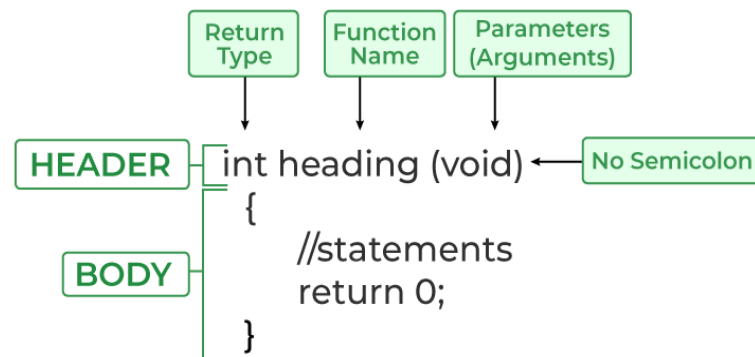
Function Definition

A function definition, on the other hand, provides the actual implementation of the function. This includes the body of the function, which contains the code that will be executed when the function is called. The definition must match the declaration in terms of name, return type, and parameters.

Syntax:

```
return_type function_name (para1_type para1_name, para2_type para2_name)
{
    // body of the function
}
```

Function Definition



```
// C program to show function call, declaration and definition
#include <stdio.h>
```

```
//Function Declaration with return type, name and parameters
int sum(int , int );
```

```
// Driver code
int main(){
    // Calling sum function and storing its value in add variable
    int add = sum(10, 30);
    printf("Sum is: %d", add);
    return 0;}
```

```
// Function that takes two parameters a and b as inputs and returns their sum
int sum(int a, int b)
{
    return a + b; }
```

Output

Sum is: 40

Function Arguments

Function arguments (also called parameters) are the values or variables passed into a function when it's called. These arguments allow a function to receive input, which can be used within the function's body to perform calculations or operations. When you declare a function, you specify the number and data type of arguments it expects within parentheses ().

Example:

```
int add(int a, int b); (This function add takes two integer arguments a and b.)
```

When you call a function, you provide the actual values for the arguments.

Example:

```
int sum = add(5, 3); (Here, 5 is passed to a, and 3 is passed to b.)
```

Types of Arguments

Actual Arguments: The values that are passed to the function during the function call. (e.g., 5 and 3 in the example above)

Formal Arguments: The parameters that are defined in the function declaration/definition. (e.g., int a and int b in the example above)

Return Type

A function can return a value back to the caller after it has finished executing. The return type is specified before the function name in the function declaration.

Example:

`int add(int a, int b);` (This declares that the function `add` will return an integer value.)

return Statement: Inside the function definition, the return statement is used to send a value back.

Example:

```
int add(int a, int b) {  
    int sum = a + b;  
    return sum; // Returns the calculated sum  
}
```

void Return Type: If a function does not need to return any value, you use the `void` keyword as the return type.

Example:

`void printMessage(char message[]);` (This function does not return anything.)

Example with Arguments and Return Type

```
#include <stdio.h>
```

```
// Function declaration
```

```
int multiply(int x, int y);
```

```
int main() {
```

```
    int num1 = 10;
```

```
    int num2 = 20;
```

```
    int result = multiply(num1, num2); // Function call with arguments
```

```
    printf("The product of %d and %d is %d\n", num1, num2, result);
```

```
    return 0;
```

```
}
```

```
// Function definition
```

```
int multiply(int x, int y) {
```

```
    return x * y; // Returns the product of x and y
```

```
}
```

Key Points

- A function can have zero or more arguments.
- A function can have only one return value. If you need to return multiple values, you can use pointers, arrays, or structures.
- If no return type is specified, C compilers assume it to be int (though it's best practice to always specify the return type explicitly).

How to Call a Function

To call a function in C, you use the function's name followed by parentheses (). If the function takes arguments, you provide the values for those arguments inside the parentheses.

```
#include <stdio.h>
```

```
// Function declaration
```

```
int add(int a, int b);
```

```
int main() {
```

```
    int num1 = 5;
```

```
    int num2 = 3;
```

```
    int sum = add(num1, num2); // Function call
```

```
    printf("The sum is: %d\n", sum);
```

```
    return 0;
```

```
}
```

```
// Function definition
```

```
int add(int a, int b) {
```

```
    return a + b;
```

```
}
```

In this example, `add(num1, num2)` is the function call.

Types of Function Calls in C:

There are two main ways to call a function in C, distinguished by how arguments are passed:

1. Call by Value

- When you call a function "by value," a copy of the actual argument's value is passed to the function's formal parameter.
- Any changes made to the formal parameter inside the function do not affect the original variable in the calling function.

Example:

```
#include <stdio.h>
```

```
void modify(int x) {  
    x = x * 2;  
    printf("Value inside function: %d\n", x);  
}
```

```
int main() {  
    int num = 10;  
    modify(num); // Call by value  
    printf("Value in main: %d\n", num);  
    return 0;  
}
```

Output:

```
// Value inside function: 20
```

```
// Value in main: 10
```

As you can see, even though x was modified inside the modify function, the value of num in main remained unchanged.

2. Call by Reference

- When you call a function "by reference," the memory address of the actual argument is passed to the function's formal parameter.
- The formal parameter acts as an alias for the actual argument. Therefore, any changes made to the formal parameter inside the function *do* affect the original variable in the calling function.
- In C, call by reference is achieved using pointers.

Example:

```
#include <stdio.h>
```

```
void modify(int *x) { // x is a pointer
    *x = (*x) * 2; // Dereference x to access the value at that address
    printf("Value inside function: %d\n", *x);
}
```

```
int main() {
    int num = 10;
    modify(&num); // Call by reference (pass the address of num)
    printf("Value in main: %d\n", num);
    return 0;
}
```

Output:

```
// Value inside function: 20
```

```
// Value in main: 20
```

Here, because we passed the address of num to the modify function, modifying *x inside the function directly changed the value of num in main.

Choosing Between Call by Value and Call by Reference

- Use **call by value** when you don't need to modify the original variable in the calling function. This is generally safer as it prevents unintended side effects.
- Use **call by reference** when you need to modify the original variable or when you need to pass large data structures (like arrays or structs) to avoid the overhead of copying.

Recursion in C

Recursion in C is a programming technique where a function calls itself either directly or indirectly to solve a problem.

Recursive functions are commonly used to solve problems that can be broken down into smaller, similar sub-problems. Each recursive call should bring the problem closer to a base case, which is the condition that terminates the recursion.

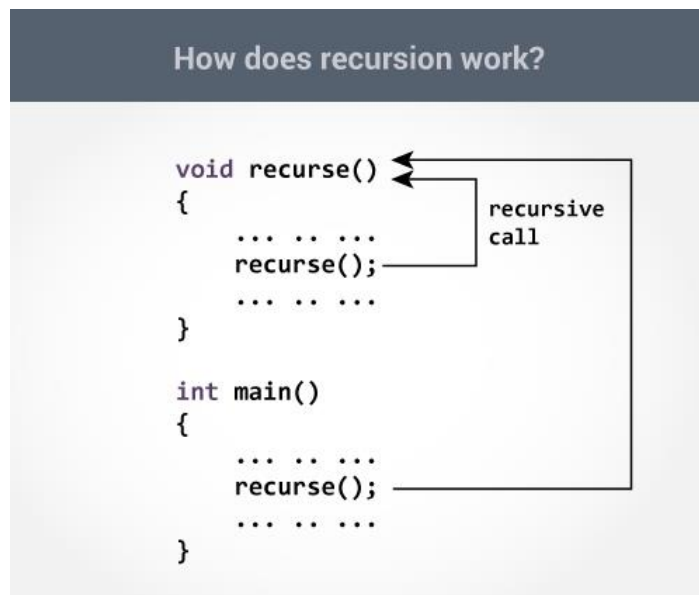
Key Components of a Recursive Function

1. **Base Case:** The condition that stops the recursion to prevent infinite recursion.

2. **Recursive Case:** The part of the function where the function calls itself with modified arguments.

General Syntax of a Recursive Function

```
return_type function_name(parameters) {  
    if (base_case_condition) {  
        // Base case logic  
        return some_value;  
    } else {  
        // Recursive case logic  
        return function_name(modified_parameters); }  
}
```



Example: Sum of Natural Numbers Using Recursion

```
#include <stdio.h>
```

```
int sum(int n);
```

```
int main() {  
    int number, result;  
    printf("Enter a positive integer: ");  
    scanf("%d", &number);
```

```
    result = sum(number);  
    printf("sum = %d", result);  
    return 0;}
```

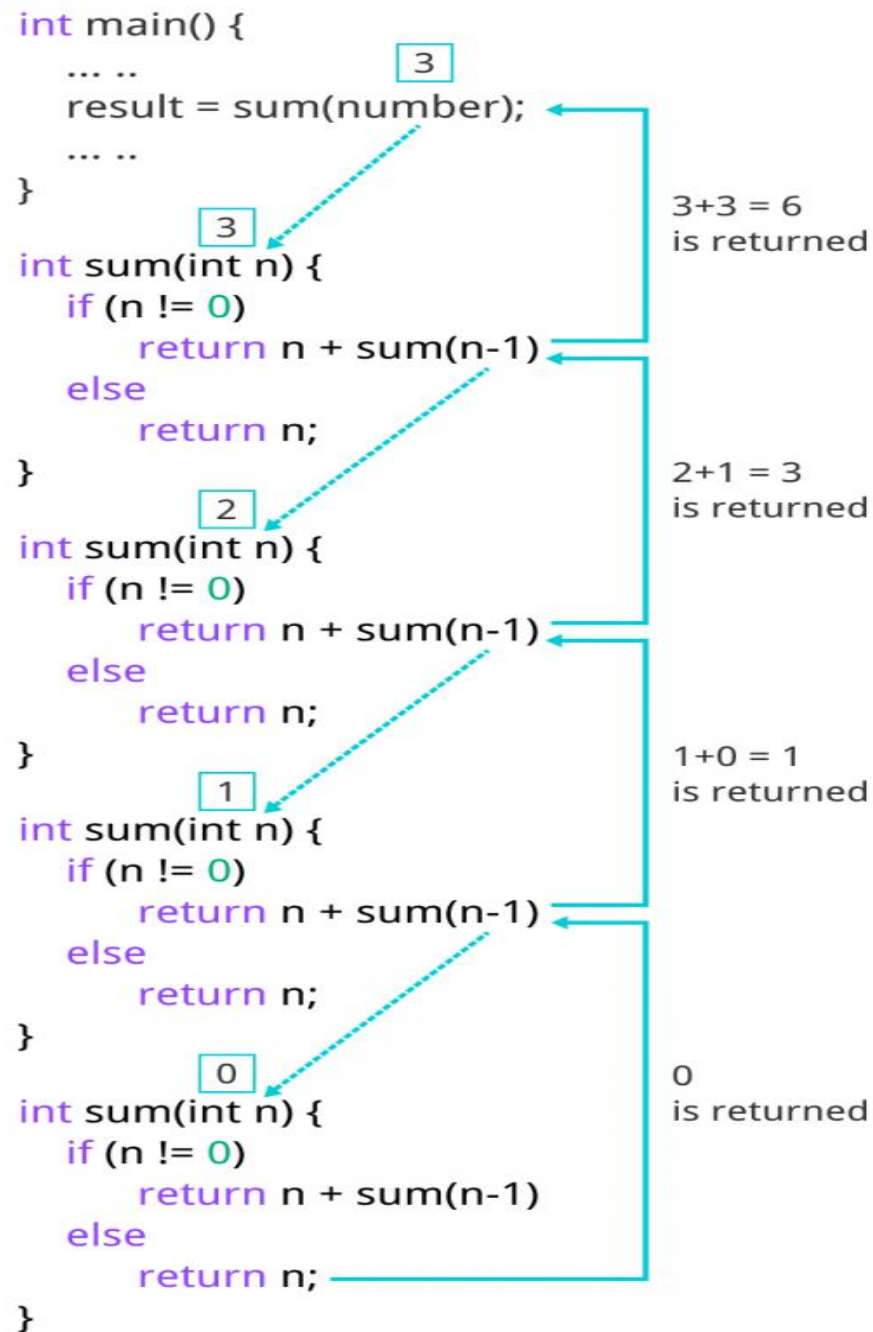
```

int sum(int n) {
    if (n != 0)
        return n + sum(n-1);
    else
        return n;}

```

Output

Enter a positive integer:3
sum = 6



Example: Factorial of a Number using recursion

```
#include <stdio.h>
```

```
int factorial(int n) {  
    if (n == 0) // Base case  
        return 1;  
    else  
        return n * factorial(n - 1); // Recursive call  
}  
  
int main() {  
    int num = 5;  
    printf("Factorial of %d is %d\n", num, factorial(num));  
    return 0;  
}
```

How it Works: Factorial is the product of all positive numbers from 1 to the given number (n). Example: Factorial of 5 = $5 \times 4 \times 3 \times 2 \times 1 = 120$

- The **base case** is $n == 0$, where the factorial of 0 is defined as 1. This stops the recursion.
- For other values of n, the function calls itself with a smaller number (n-1), multiplying n with the result of the smaller problem.

Step-by-Step Execution for factorial (5):

1. factorial(5) calls $5 * \text{factorial}(4)$
2. factorial(4) calls $4 * \text{factorial}(3)$
3. factorial(3) calls $3 * \text{factorial}(2)$
4. factorial(2) calls $2 * \text{factorial}(1)$
5. factorial(1) calls $1 * \text{factorial}(0)$
6. factorial(0) returns 1 (base case)
7. Now the results are multiplied step-by-step:
 - $1 * 1 = 1$
 - $2 * 1 = 2$
 - $3 * 2 = 6$
 - $4 * 6 = 24$
 - $5 * 24 = 120$

Output:

Factorial of 5 is 120

Example: Fibonacci Series using recursion

```
#include <stdio.h>
int fibonacci(int n) {
    if (n == 0) // Base case
        return 0;
    else if (n == 1) // Base case
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2); // Recursive call
}
int main() {
    int terms = 5;
    printf("Fibonacci Series: ");
    for (int i = 0; i < terms; i++) {
        printf("%d ", fibonacci(i));
    }
    return 0;}
```

How it Works: The **Fibonacci series** is a sequence of numbers where each number is the sum of the two preceding numbers. Example: 0, 1, 1, 2, 3, 5, 8, 13 ...

1. The **base cases** are:
 - fibonacci(0) returns 0.
 - fibonacci(1) returns 1.
2. For larger numbers, the function calculates the sum of fibonacci(n-1) and fibonacci(n-2).

Step-by-Step Execution for fibonacci(5):

1. fibonacci(5) calls fibonacci(4) + fibonacci(3)
2. fibonacci(4) calls fibonacci(3) + fibonacci(2)
3. fibonacci(3) calls fibonacci(2) + fibonacci(1)
4. This process continues until it reaches the base cases (fibonacci(0) and fibonacci(1)).
5. The results are then added together step-by-step to get the Fibonacci number for n=5.

Output:

Fibonacci Series:

0 1 1 2 3 5 8 13 21 34

Passing Array to Function:

In C, passing an array to a function is done by passing a pointer to the first element of the array. Since arrays in C are passed by reference, modifications to the array inside the function will affect the original array.

There are two common ways to pass arrays to functions in C:

1. Passing an Array (as a Pointer) to a Function

You pass the array's name, which is actually a pointer to the first element of the array, to the function.

Example:

```
#include <stdio.h>

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]); // Calculate the size of the array
    printArray(arr, size); // Passing the array to the function
    return 0;
}
```

In this example:

- `arr[]` in the function parameter list is equivalent to a pointer to the first element of the array.
- We also pass the array's size because the function needs to know how many elements are in the array.

2. Passing an Array Using a Pointer

You can explicitly declare the array parameter as a pointer in the function definition. This is essentially the same as the first method but uses a more explicit pointer notation.

Example

```
#include <stdio.h>
```

```
void printArray(int *arr, int size) {  
    for (int i = 0; i < size; i++) {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");}
```

```
int main() {  
    int arr[] = {10, 20, 30, 40, 50};  
    int size = sizeof(arr) / sizeof(arr[0]);  
    printArray(arr, size); // Passing the array to the function  
    return 0;}
```

Here:

- The function parameter `int *arr` indicates that the function is expecting a pointer to an integer (which points to the first element of the array).
- The size of the array is passed as an argument to avoid potential issues with array bounds inside the function.

Passing a 2D Array to a Function

A 2D array in C is essentially an array of arrays. When passing a 2D array to a function, you have to specify the second dimension (number of columns), but the first dimension can be omitted.

Method 1: Using a Pointer to an Array of Arrays

For 2D arrays, you can pass them by specifying the number of columns explicitly in the function signature.

Example:

```
#include <stdio.h>
```

```
void print2DArray(int arr[][3], int rows) {  
    for (int i = 0; i < rows; i++) {  
        for (int j = 0; j < 3; j++) {  
            printf("%d ", arr[i][j]); }  
        printf("\n"); } }
```

```
int main() {
    int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int rows = 2;
    print2DArray(arr, rows); // Passing the 2D array to the function
    return 0;}

```

Explanation:

- arr[][3] in the function parameter specifies that arr is a 2D array where the second dimension has 3 elements.
- The first dimension (number of rows) is determined at runtime or passed as an argument.

Method 2: Using Pointer Notation for 2D Arrays

Another way to pass a 2D array is to use a pointer that represents the array. This requires explicitly defining the column size.

Example:

```
#include <stdio.h>

```

```
void print2DArray(int (*arr)[3], int rows) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }
}

```

```
int main() {
    int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int rows = 2;

    print2DArray(arr, rows); // Passing the 2D array to the function

    return 0;
}

```

Explanation:

- `int (*arr)[3]` is a pointer to an array of 3 integers. This notation is necessary because C doesn't know the size of the second dimension unless it's explicitly mentioned.

Important Notes:

1. **Size of Array:** C does not know the size of the array when it is passed to a function. It only knows the address of the first element (the pointer). Therefore, it is common practice to also pass the size of the array to the function to avoid out-of-bounds access.

Storage classes:

In C, **storage classes** define the **lifetime**, **scope**, and **visibility** of variables. They determine how variables are stored and accessed in memory. There are **4 primary storage classes** in C:

1. auto

The auto storage class is the default storage class for local variables inside functions. It means that the variable is automatically created when the function is called and destroyed when the function exits.

The auto storage class is implicit, so you don't have to explicitly declare a variable with auto it's assumed for local variables unless specified otherwise.

- **Scope:** Local to the function/block in which it is declared.
- **Lifetime:** The duration of the function/block execution.
- **Initialization:** Not initialized by default (contains garbage value if not initialized).

Example:

```
#include <stdio.h>
```

```
void function() {  
    auto int x = 10; // Equivalent to just 'int x = 10;'  
    printf("%d\n", x); // Prints 10  
}
```

```
int main() {  
    function();  
    // x is not accessible here  
    return 0;}
```

2. register

The register storage class is used to suggest to the compiler that the variable should be stored in a processor register for faster access, rather than in RAM. However, this is just a suggestion the compiler may ignore it if no registers are available.

- **Scope:** Local to the function/block in which it is declared.
- **Lifetime:** The duration of the function/block execution.
- **Initialization:** Not initialized by default(contains garbage value if not initialized).

Example:

```
#include <stdio.h>
```

```
void function() {  
    register int counter = 0;  
    for (counter = 0; counter < 5; counter++) {  
        printf("%d ", counter); // Prints numbers from 0 to 4  
    }  
}
```

```
int main() {  
    function();  
    return 0;  
}
```

3. static

The static storage class is used to preserve the value of a variable across multiple function calls. A static variable is initialized only once and retains its value between function calls. Static variables can be declared inside functions or outside functions (global variables).

- **Scope:**
 - If declared inside a function: Local to the function, but retains its value across function calls.
 - If declared outside a function: The variable has **global scope** but is only accessible within the file (it has **file scope**).
- **Lifetime:** The variable exists for the lifetime of the program.
- **Initialization:** Initializes only once, and retains the value between calls (contains 0 if not initialized).

Example:

```
#include <stdio.h>
```

```
void function() {  
    static int count = 0; // Initialized only once  
    count++;  
    printf("%d ", count); // Prints incremented value across function calls  
}
```

```
int main() {  
    for (int i = 0; i < 5; i++) {  
        function(); // Prints 1 2 3 4 5  
    }  
    return 0;}
```

4. extern

The extern storage class is used to declare a variable or function that is defined in another file or elsewhere in the same file. It is typically used for global variables and functions that are shared between different files in a program.

- **Scope:** Global, accessible throughout the program, across multiple files if declared correctly.
- **Lifetime:** Exists for the lifetime of the program.
- **Initialization:** No initialization (it simply refers to a variable that has been declared elsewhere/ contains 0 if it's not linked with file that contains said variables).

Example:

```
// file1.c  
#include <stdio.h>  
int count = 10; // Global variable defined in file1.c  
int main() {  
    count++;  
    printf("%d ", count);  
    return 0;  
}
```

```
// file2.c
#include<stdio.h>
#include "file1.c"
extern int count; // Declaring the external variable
int main() {
    printf("%d",count); // prints 10
    return 0;}
```

In this example:

- The extern keyword in file2.c tells the compiler that count is defined in another file (file1.c).
- The variable count is global, and its value is shared between the files.

Types of Variables

1. Local Variables

Local variables are declared within a function or a block and can only be accessed within that function or block. They are created when the function is called and destroyed when the function exits.

Example:

```
#include <stdio.h>
void myFunction() {
    int localVar = 10; // Local variable
    printf("Local Variable: %d\n", localVar);
}

int main() {
    myFunction();
    printf("%d", localVar); // This would cause an error
    return 0; }
```

2. Global Variables

Global variables are declared outside of all functions and can be accessed by any function within the same file or other files if declared with the extern keyword. They exist for the lifetime of the program.

Example:

```
#include <stdio.h>

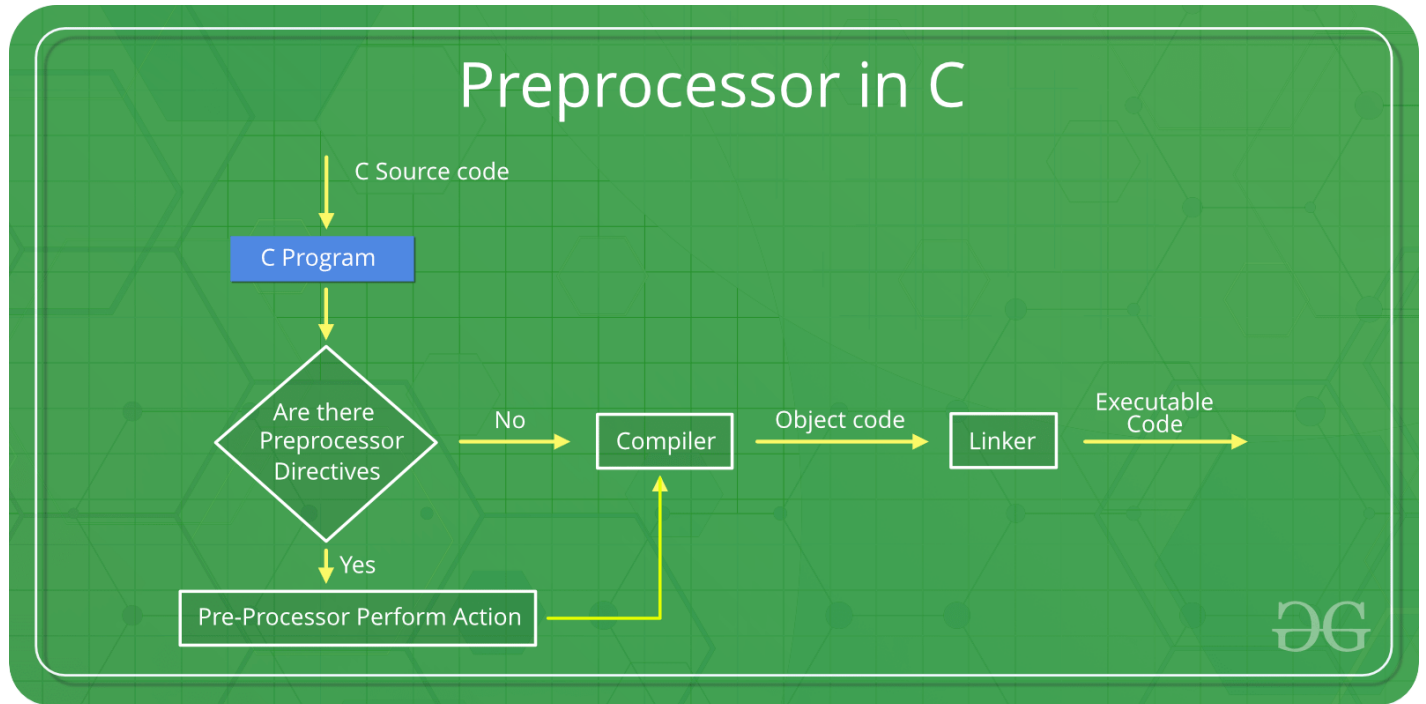
int globalVar = 20; // Global variable

void myFunction() {
    printf("Global Variable: %d\n", globalVar);}

int main() {
    myFunction();
    printf("Global Variable in main: %d\n", globalVar);
    return 0;}
```

C Preprocessors:

Preprocessors are programs that process the source code before compilation. Several steps are involved between writing a program and executing a program in C.



You can see the intermediate steps in the above diagram. The source code written by programmers is first stored in a file, let the name be “**program.c**“. This file is then processed by preprocessors and an expanded source code file is generated named “program.i”.

This expanded file is compiled by the compiler and an object code file is generated named “program.obj”. Finally, the linker links this object code file to the object code of the library functions to generate the executable file “program.exe”.

Preprocessor Directives in C

Preprocessor programs provide preprocessor directives that tell the compiler to preprocess the source code before compiling.

All of these preprocessor directives begin with a ‘#’ (hash) symbol. The ‘#’ symbol indicates that whatever statement starts with a ‘#’ will go to the preprocessor program to get executed. We can place these preprocessor directives anywhere in our program.

Examples of some preprocessor directives are: *#include*, *#define*, *#ifndef*, etc.

Note Remember that the # symbol only provides a path to the preprocessor, and a command such as include is processed by the preprocessor program. For example, #include will include the code or content of the specified file in your program.

Types of C Preprocessors

There are 4 Main Types of Preprocessor Directives:

1. Macros
2. File Inclusion
3. Conditional Compilation
4. Other directives

1. Macros

In C, Macros are pieces of code in a program that is given some name. Whenever this name is encountered by the compiler, the compiler replaces the name with the actual piece of code. The ‘**#define**’ directive is used to define a macro.

Syntax of Macro Definition

#define token value

Where after preprocessing, the *token* will be expanded to its *value* in the program.

Example of Macro

```
#include <stdio.h>
```

```
// macro definition
```

```
#define LIMIT 5
```

```
int main()
{
    for (int i = 0; i < LIMIT; i++) {
        printf("%d \t", i);
    }
    return 0;}

```

Output

0 1 2 3 4

In the above program, when the compiler executes the word LIMIT, it replaces it with 5. The word ‘**LIMIT**’ in the macro definition **is called a macro template** and ‘**5**’ **is macro expansion**.

Note There is no semi-colon (;) at the end of the macro definition. Macro definitions do not need a semi-colon to end.

Macros with Arguments

We can also pass arguments to macros. Macros defined with arguments work similarly to functions.

Example

```
#define foo(a, b) a + b
```

```
#define func(r) r * r
```

```
// C Program to illustrate function like macros
```

```
#include <stdio.h>
```

```
// macro with parameter
```

```
#define AREA(L, B) (L * B)
```

```
int main(){
```

```
    int L = 10, B = 5, area;
```

```
    area = AREA(l1, l2);
```

```
    printf("Area of rectangle is: %d", area);
```

```
    return 0;}
```

Output

Area of rectangle is: 50

We can see from the above program that whenever the compiler finds AREA(L, B) in the program, it replaces it with the statement (L*B). Not only this, but the values passed to the macro template AREA(B, B) will also be replaced in the statement (L*B). Therefore AREA(10, 5) will be equal to 10*5.

2. File Inclusion

This type of preprocessor directive tells the compiler to include a file in the source code program. The **#include preprocessor directive** is used to include the header files in the C program.

There are two types of files that can be included by the user in the program:

a. Standard Header Files

The standard header files contain definitions of pre-defined functions like **printf()**, **scanf()**, etc. These files must be included to work with these functions. Different functions are declared in different header files.

For example, standard I/O functions are in the 'stdio' file whereas functions that perform string operations are in the 'string' file.

Syntax

```
#include <file_name>
```

where *file_name* is the name of the header file to be included. The ‘<’ and ‘>’ **brackets** tell the compiler to look for the file in the **standard directory**.

b. User-defined Header Files

When a program becomes very large, it is a good practice to divide it into smaller files and include them whenever needed. These types of files are user-defined header files.

Syntax

```
#include "filename"
```

The **double quotes (" ")** tell the compiler to search for the header file in the **source file's directory**.

3. Conditional Compilation

Conditional Compilation in C directives is a type of directive that helps to compile a specific portion of the program or to skip the compilation of some specific part of the program based on some conditions. There are the following preprocessor directives that are used to insert conditional code:

1. **#if Directive**
2. **#ifdef Directive**
3. **#ifndef Directive**
4. **#else Directive**
5. **#elif Directive**
6. **#endif Directive**

Syntax

```
#ifdef macro_name
```

```
    // Code to be executed if macro_name is defined
```

```
#ifndef macro_name
```

```
    // Code to be executed if macro_name is not defined
```

```
#if constant_expr
```

```
    // Code to be executed if constant_expression is true
```

```
#elif another_constant_expr
```

```
    // Code to be executed if another_constant_expression is true
```

```
#else
```

```
    // Code to be executed if none of the above conditions are true
```

```
#endif
```

Example

The below example demonstrates the use of #include #if, #elif, #else, and #endif preprocessor directives.

```
//program to demonstrates the use of #if, #elif, #else and #endif preprocessor directives.
```

```
#include <stdio.h>
```

```
// defining PI
```

```
#define PI 3.14159
```

```
int main(){
```

```
#ifdef PI
```

```
    printf("PI is defined\n");
```

```
#elif defined(SQUARE)
```

```
    printf("Square is defined\n");
```

```
#else
```

```
#error "Neither PI nor SQUARE is defined"
```

```
#endif
```

```
#ifndef SQUARE
```

```
    printf("Square is not defined");
```

```
#else
```

```
    printf("Square is defined");
```

```
#endif
```

```
    return 0;}
```

Output

PI is defined

Square is not defined

4. Other Directives

Apart from the above directives, there are two more directives that are not commonly used. These are:

1. **#undef Directive**
2. **#pragma Directive**

1. #undef Directive

The #undef directive is used to undefine an existing macro. This directive works as:

```
#undef LIMIT
```

Using this statement will undefine the existing macro LIMIT. After this statement, every “#ifdef LIMIT” statement will evaluate as false.

Example

The below example demonstrates the working of #undef Directive.

```
#include <stdio.h>
```

```
// defining MIN_VALUE
```

```
#define MIN_VALUE 10
```

```
int main() {
```

```
printf("Min value is: %d\n",MIN_VALUE);
```

```
//undefining max value
```

```
#undef MIN_VALUE
```

```
// again redefining MIN_VALUE
```

```
#define MIN_VALUE 20
```

```
printf("Min value after undef and again redefining it: %d\n", MIN_VALUE);  
return 0;}
```

Output

```
Min value is: 10
```

```
Min value after undef and again redefining it: 20
```

2. #pragma Directive

This directive is a special purpose directive and is used to turn on or off some features. These types of directives are compiler-specific, i.e., they vary from compiler to compiler.

Syntax

#pragma directive

Some of the #pragma directives are discussed below:

1. **#pragma startup:** These directives help us to specify the functions that are needed to run before program startup (before the control passes to main()).
2. **#pragma exit:** These directives help us to specify the functions that are needed to run just before the program exit (just before the control returns from main()).

List of preprocessor directives in C

Preprocessor Directives	Description
#define	Used to define a macro
#undef	Used to undefine a macro
#include	Used to include a file in the source code program
#ifdef	Used to include a section of code if a certain macro is defined by #define
#ifndef	Used to include a section of code if a certain macro is not defined by #define
#if	Check for the specified condition
#else	Alternate code that executes when #if fails
#elif	Combines else and if for another condition check
#endif	Used to mark the end of #if, #ifdef, and #ifndef

